Reverse Engineering Report

Python Executable — password_protected.exe

University Assignment | Nour Mohamed - 202302744

Overview

This report documents the complete step-by-step process used to reverse engineer a protected Python executable (password_protected.exe). The binary was protected using three layers of protection that had to be bypassed in sequence to recover the original Python source code.

Protection Layers Identified

Layer	Tool	Purpose
Outer	UPX 4.24 (LZMA, brute)	Compressed the entire executable
Middle	Nuitka onefile	Packaged Python runtime + script into one exe
Inner	PyArmor	Encrypted the Python source bytecode

Step-by-Step Process

Step 1 — Run the Executable and Observe Output

The executable was run to observe its behavior and capture the exact program output before any analysis tools were used.

Commands tested:

```
Enter the password: g
Access denied
Enter the password: python123
Access granted
Press any key to continue...
```

Result: Confirmed the program is a simple password checker with no attempt limit.

Step 2 — Open in Detect-It-Easy (DIE)

The executable was dragged onto Detect-It-Easy to fingerprint the binary and identify any packers or protections.

Property	Value
Packer	UPX 4.24 [LZMA, brute]
Compiler	Microsoft Visual C/C++ 19.50 (LTCG/C)
Language	C
Library	Microsoft C/C++ Runtime (dynamic)
Protection	Generic (Section #0 has RWX + IAT empty)

Result: UPX packer confirmed. The binary needed to be unpacked before further analysis.

Step 3 — Unpack UPX

UPX was downloaded and placed in the same folder as the executable. The following command was run in PowerShell:

```
& "C:\Users\nourr\Downloads\upx-5.1.1-win64 (1)\upx-5.1.1-win64\upx.exe" -d password_protected.exe -o password_unpacked.exe
```

Result: password_unpacked.exe created. The UPX layer was removed successfully.

Step 4 — Re-analyze Unpacked Binary in DIE

The unpacked binary was re-opened in DIE. The UPX packer signature was gone. DIE now showed a high-entropy .rsrc section, confirming embedded Python files inside the executable.

Result: High entropy .rsrc section confirmed embedded Python files. Next step was to identify the Python packager.

Step 5 — Identify Python Packager via Strings Analysis

Sysinternals strings.exe was run on the unpacked binary to extract all readable strings:

```
.\strings.exe password_unpacked.exe > unpacked_strings.txt
```

The output file was opened and searched for known packager signatures. The string NUITKA_ONEFILE_PARENT was found, confirming the binary was packaged with Nuitka onefile — not PyInstaller or cx_Freeze.

Step 5b — Password Recovery via Memory Dump (System Informer)

After identifying the Nuitka packaging, System Informer was used to perform a full memory dump of the running process to find the decrypted password in memory.

Process:

- Ran password_protected.exe and kept it waiting at the password prompt
- Opened System Informer and located the password_protected.exe process

- Right-clicked the process -> Create dump file -> Full dump
- Saved the dump file to disk
- Opened the dump file in a hex editor / strings tool and searched for readable strings

The following plaintext strings were found inside the memory dump, fully decrypted by PyArmor at runtime:

```
PYARMOR
password
input
user_input
print
__armor_wrap__
<frozen password>
<module>
python123
Enter the password:
Access granted
Access denied
Press any key to continue...
pytransform.dll
PyInit_pytransform
```

Result: The password python123 was found in plaintext inside the memory dump. This confirmed the password before the sys.settrace hook was even run. The dump also revealed all variable names, string constants, and the PyArmor DLL signature.

Step 6 — Extract Files by Running the Executable

Nuitka onefile extracts itself to a temp folder at runtime. The exe was run in one PowerShell window and kept waiting at the password prompt. In a second window, the temp folder was searched immediately:

```
ls C:\Users\nourr\AppData\Local\Temp\ -Recurse | where {$_.Name -like
"password.py"} | Sort-Object LastWriteTime -Descending
```

The file was found and copied to the Desktop before it could be deleted:

```
$f = (ls C:\Users\nourr\AppData\Local\Temp\ -Recurse | where {$_.Name -like
"password.py"} | Sort-Object LastWriteTime -Descending | Select-Object -First
1).FullName; copy $f C:\Users\nourr\Desktop\
```

Result: password.py copied to Desktop. File was 1,656 bytes.

Step 7 — Inspect the Extracted password.py

The extracted file was opened in Notepad. The content was not readable Python — it showed encrypted bytes:

```
from pytransform import pyarmor
pyarmor(__name__, __file__, b'\x50\x59\x41\x52\x4d\x4f\x52...', 2)
```

Result: PyArmor confirmed as the inner protection layer. Source was encrypted and could not be read directly.

Step 8 — Identify the Python Version

While the exe was running, the Nuitka extraction folder was checked for the Python DLL version:

```
ls C:\Users\nourr\AppData\Local\Temp\ -Recurse | where {$_.Name -like "*.dll"}
| Sort-Object LastWriteTime -Descending | Select-Object -First 5
```

Output showed python39.dll, confirming the exe uses Python 3.9 internally. This is critical — PyArmor's pytransform.pyd only works with the matching Python version.

Step 9 — Install Python 3.9

The system had Python 3.13 and 3.14 which are incompatible with the embedded pytransform.pyd. Python 3.9.13 was downloaded and installed from python.org. After install, verified with:

```
py -0
-V:3.14 *    Python 3.14 (64-bit)
-V:3.13      Python 3.13 (64-bit)
-V:3.9       Python 3.9 (64-bit)
```

Result: Python 3.9 installed and accessible via the py launcher.

Step 10 — Create the Runtime Hook Script

A hook script (hook.py) was created on the Desktop. This script intercepts Python execution at runtime using sys.settrace, capturing decrypted bytecode as PyArmor decrypts it in memory:

```
import sys
import dis

def trace(frame, event, arg):
    if event == 'call':
        code = frame.f_code
        if 'password' in code.co_filename.lower() or 'frozen' in
str(code.co_filename).lower():
            print('=== RECOVERED ===')
            print(f'File: {code.co_filename}')
            print(f'Constants: {code.co_consts}')
            print(f'Variables: {code.co_varnames}')
            print(f'Names: {code.co_names}')
            try:
                for instr in dis.Bytecode(code):
                    print(instr)
            except Exception as e:
                print(f'dis error: {e}')
    return trace
```

```
sys.settrace(trace)
exec(open('password.py').read())
```

Purpose: sys.settrace intercepts every function call. When PyArmor decrypts the code at runtime, the hook captures the decrypted co_names and co_varnames before execution.

Step 11 — Run the Hook from the PyArmor Temp Folder

The hook had to run from the folder where pytransform.pyd and its dependencies existed. The exe was run again in Window 1, then in Window 2:

```
$folder = (ls C:\Users\nourr\AppData\Local\Temp\ -Recurse | where {$_.Name -
like 'password.py'} | Sort-Object LastWriteTime -Descending | Select-Object -
First 1).DirectoryName
copy C:\Users\nourr\Desktop\hook.py $folder
cd $folder
py -3.9 hook.py
```

Result: Hook executed successfully. PyArmor loaded pytransform.pyd and decrypted the code at runtime.

Step 12 — Extract Variable Names from Hook Output

The hook output contained thousands of lines from Python internals. Scrolling to the very bottom revealed the entry for <frozen password>:

```
=== RECOVERED ===
File: <frozen password>
Names: ('password', 'input', 'user_input', 'print', '__armor_wrap__')
```

Name	Role
password	Variable holding the hardcoded password string
input	Built-in input() function for user prompt
user_input	Variable storing what the user types
print	Built-in print() function for output
__armor_wrap__	PyArmor runtime wrapper (not original code)

Result: 100% confirmed variable names recovered from decrypted bytecode.

Step 13 — Reconstruct the Source Code

Combining the recovered variable names, the program output observed in Step 1, and the password recovered from the hook output, the original source code was reconstructed:

```
password = "python123"
user_input = input("Enter the password: ")
if user_input == password:
    print("Access granted")
```

```
else:
    print("Access denied")
input("Press any key to continue...")
```

The reconstructed code was tested and produced identical output to the original executable, confirming a successful recovery.

Summary of Findings

Finding	Value	Method Used
Password	python123	Memory dump (System Informer) + runtime hook
Original filename	password.py	Nuitka temp folder extraction
Variable: password	password	sys.settrace hook on bytecode
Variable: user input	user_input	sys.settrace hook on bytecode
Python version	3.9	python39.dll in Nuitka temp folder
Inner protection	PyArmor	pyarmor() call in extracted .py file
Middle layer	Nuitka onefile	NUITKA_ONEFILE_PARENT string in binary
Outer layer	UPX 4.24	Detect-It-Easy fingerprint

Tools Used

Tool	Purpose	Result
System Informer	Full process memory dump	Found python123 and all strings in plaintext
Detect-It-Easy (DIE)	Binary fingerprinting	Identified UPX packer
UPX 5.1.1	Decompress UPX layer	Produced password_unpacked.exe
Sysinternals Strings	String extraction	Found NUITKA_ONEFILE_PARENT
PowerShell	File monitoring during runtime	Captured temp folder files
Python 3.9 + sys.settrace	Runtime bytecode interception	Recovered variable names